

4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem Statement

The pharmaceutical market experiences frequent medicine price fluctuations due to changes in manufacturing costs, currency exchange rates, supply chain conditions, and market demand. Patients, pharmacists, and healthcare providers often struggle to track these changes or identify affordable alternatives efficiently.

Most available pharmacy systems provide only current medicine information without maintaining historical pricing records or offering predictive insights into future prices. Additionally, users seeking medicine information must manually search across multiple sources, making the process time-consuming and inconvenient.

To address these limitations, this project proposes a comprehensive Medicine Price Prediction System that integrates medicine search, historical price tracking, intelligent price prediction, affordable alternative recommendations, analytics dashboards, and an AI-powered RAG chatbot capable of answering pharmaceutical questions using trusted medical knowledge.

Project Objectives

The primary objectives of the proposed system are:

- Develop a centralized medicine information platform.
- Retrieve medicine information from external pharmaceutical APIs.
- Store medicine information locally for faster retrieval.
- Maintain historical price records.
- Forecast future medicine prices using Artificial Intelligence.
- Recommend cheaper therapeutic alternatives.
- Provide interactive analytics dashboards.
- Integrate a Retrieval-Augmented Generation (RAG) chatbot for pharmaceutical assistance.
- Ensure scalability, maintainability, and high availability.

4.2 Use Case Diagram & Descriptions

Actors

Primary Actor

- Public User

Secondary Actors

- External Drug API
- RAG Knowledge Base
- PostgreSQL Database

User Use Cases

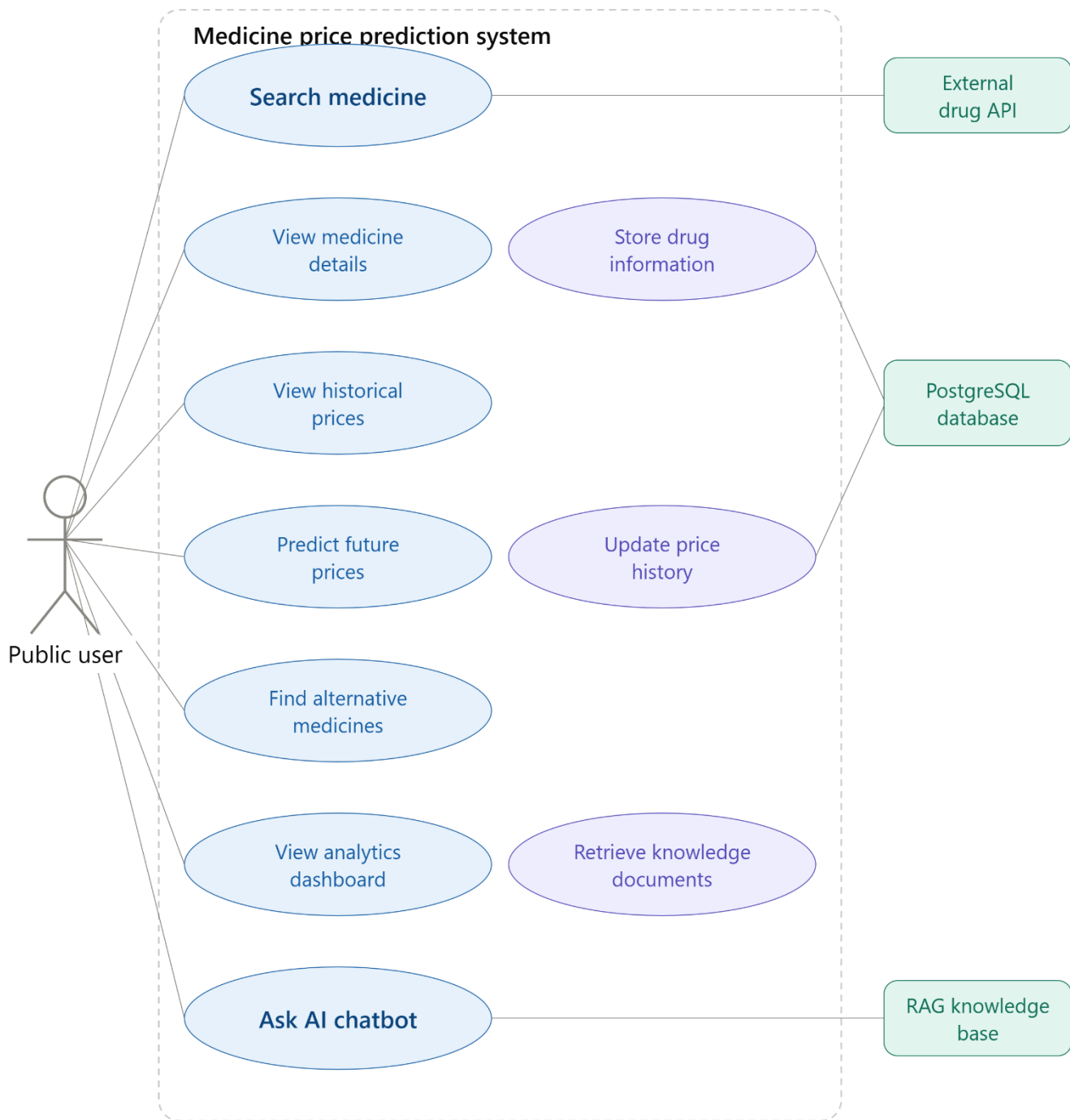
The Public User can:

- Search medicines
- View medicine details
- View historical prices
- Predict future prices
- View price trend charts
- Find cheaper alternatives
- Ask questions using the AI chatbot
- View analytics dashboards

The backend system automatically:

- Retrieves external API data
- Stores retrieved medicine information
- Updates historical prices
- Retrieves RAG knowledge
- Generates AI responses

Use Case Diagram



4.3 Functional Requirements

Medicine Management

- Search medicines by name.
- Search by manufacturer.
- Display detailed medicine information.
- Retrieve information from external APIs.
- Cache medicine information locally.
- Store historical prices.
- Prevent duplicate medicine records.
- Maintain audit logs of API responses.

Price Prediction

- Generate future price forecasts.
- Display confidence intervals.
- Allow prediction periods to be customized.
- Visualize prediction trends.

Alternative Recommendation

- Recommend cheaper medicines.
- Display medicine comparison cards.

Analytics

- Show manufacturer distribution.
- Display medicine classes.
- Display administration routes.
- Show price distributions.
- Display dashboard statistics.

AI Chatbot (RAG)

- Answer medicine-related questions.
- Retrieve relevant documents using semantic search.
- Generate contextual responses using an LLM.
- Reduce hallucinations through Retrieval-Augmented Generation.
- Provide evidence-based responses.

4.4 Non-Functional Requirements

Performance

- API response time below 2 seconds for cached data.
- Support concurrent users.
- Fast retrieval through indexed PostgreSQL queries.

Reliability

- Transaction rollback on failures.
- Persistent local caching.
- Historical data preservation.

Security

- Environment variable configuration.
- API key protection.
- Input validation.
- Secure database transactions.

Scalability

- Modular layered architecture.
- Independent AI components.
- Easily extendable APIs.

Maintainability

- Separation of concerns.
- Repository pattern.
- Service-oriented architecture.

Usability

- Responsive React interface.
- Search and filtering.
- Interactive dashboards.

4.5 Software Architecture

The project follows a layered architecture.

Presentation Layer

React + Vite

Responsibilities:

- User Interface
- Dashboard
- Charts
- Search
- Prediction Visualization
- Chatbot Interface

API Layer: FastAPI Routers

Responsibilities:

- HTTP Requests
- Input Validation
- Endpoint Routing

Service Layer

Responsibilities:

- Business Logic
- Drug Search
- Prediction
- Analytics
- RAG Query Processing

Repository Layer

Responsibilities:

- Database CRUD
- Transactions
- Query Optimization

Database Layer

PostgreSQL, Stores:

- Drugs
- Price History
- Raw API Responses

AI Layer

Contains two independent AI modules.

Price Prediction Module

- Historical price preprocessing
- Dataset cleaning
- Time-series forecasting
- Model inference

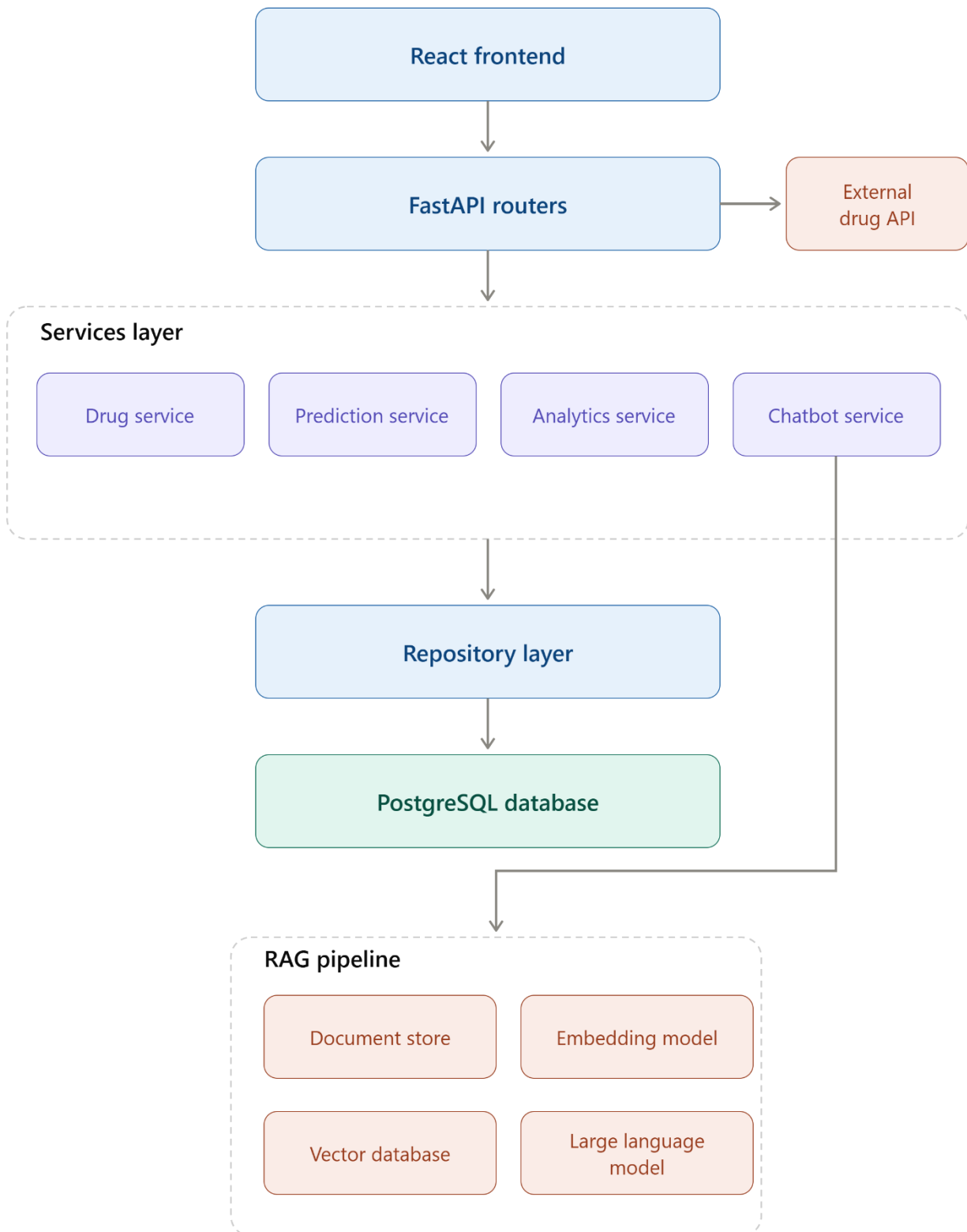
RAG Chatbot Module

- Document ingestion
- Text chunking
- Embedding generation
- Vector search
- Context retrieval
- LLM response generation

External Systems

- External Drug API
- LLM Provider
- Embedding Model

Software Architecture



4.6 Database Design & Data Modeling

ER Diagram The database contains three primary entities.

Drug Stores medicine information.

Attributes include:

- id
- commercial_name_en
- commercial_name_ar
- scientific_name
- manufacturer
- route
- drug_class
- created_at
- updated_at

Composite Unique Key: (commercial_name_en, manufacturer, route)

PriceHistory Stores historical prices.

Attributes:

- id
- drug_id
- price_egp
- recorded_at

Relationship: One Drug → Many PriceHistory records

RawApiResponse Stores complete JSON responses.

Attributes:

- id
- drug_id
- response_json
- retrieved_at

Relationship: One Drug → Many RawApiResponse records

DRUG			
int	id	PK	
string	commercial_name_en	UK	composite unique w/ manufacturer, route
string	commercial_name_ar		
string	scientific_name		
string	manufacturer	UK	
string	route	UK	
string	drug_class		
timestamp	created_at		
timestamp	updated_at		



PRICE_HISTORY		
int	id	PK
int	drug_id	FK
decimal	price_egp	
timestamp	recorded_at	



RAW_API_RESPONSE		
int	id	PK
int	drug_id	FK
json	response_json	
timestamp	retrieved_at	

4.7 Logical & Physical Schema

The Medicine Price Prediction System uses a relational PostgreSQL database designed to ensure data integrity, eliminate redundancy, and efficiently manage historical drug price records. The database follows normalization principles while remaining optimized for analytical and machine learning operations.

Primary Keys

Each table contains a primary key that uniquely identifies every record.

Table	Primary Key	Description
drugs	id (UUID)	Uniquely identifies each medicine. UUIDs provide globally unique identifiers suitable for distributed systems.
price_history	id (Integer)	Auto-incrementing identifier for each recorded price entry.
raw_api_responses	id (Integer)	Auto-incrementing identifier for each stored external API response.

Using primary keys ensures that every record can be uniquely referenced while preventing duplicate identifiers.

Foreign Keys

Foreign keys establish relationships between related tables and maintain referential integrity.

Child Table	Foreign Key	References
price_history	drug_id	drugs.id
raw_api_responses	drug_id	drugs.id

These relationships ensure that historical prices and raw API responses cannot exist without an associated medicine record. If a referenced drug does not exist, the database prevents insertion of related records.

Indexes

Indexes are created to improve query performance, particularly for search operations and time-series analysis.

Recommended indexes include:

Column	Purpose
drugs.id	Fast primary key lookups.
price_history.drug_id	Efficient retrieval of a medicine's price history.
price_history.recorded_at	Accelerates chronological sorting and forecasting queries.
raw_api_responses.drug_id	Quickly retrieves stored API responses for auditing purposes.

These indexes significantly reduce query execution time when retrieving medicine information, displaying historical prices, or preparing data for forecasting models.

Composite Unique Constraint

To prevent duplicate medicine records, the **drugs** table uses a composite unique constraint consisting of: (commercial_name_en, manufacturer, route)

This combination uniquely identifies a medicine regardless of how many times it is retrieved from the external API. If the same medicine appears in multiple searches, the system updates its historical price records instead of creating duplicate entries.

Third Normal Form (3NF)

The database is designed according to the principles of **Third Normal Form (3NF)** to reduce redundancy and improve data consistency.

The normalization process ensures that:

- Each table stores information about a single entity.
- Every non-key attribute depends only on the primary key.
- There are no transitive dependencies between non-key attributes.
- Historical price data is separated from medicine information to avoid repeated storage of medicine details.
- Raw API responses are stored independently for auditing and debugging without affecting operational data.

This normalized structure minimizes storage duplication while simplifying maintenance and future system expansion.

Optimized Storage for Time-Series Price History

Since medicine prices change over time, the system stores historical price records in a dedicated **price_history** table instead of repeatedly updating a single price field.

Each historical record contains:

- Medicine identifier (drug_id)
- Recorded price (price_egg)
- Timestamp of recording (recorded_at)

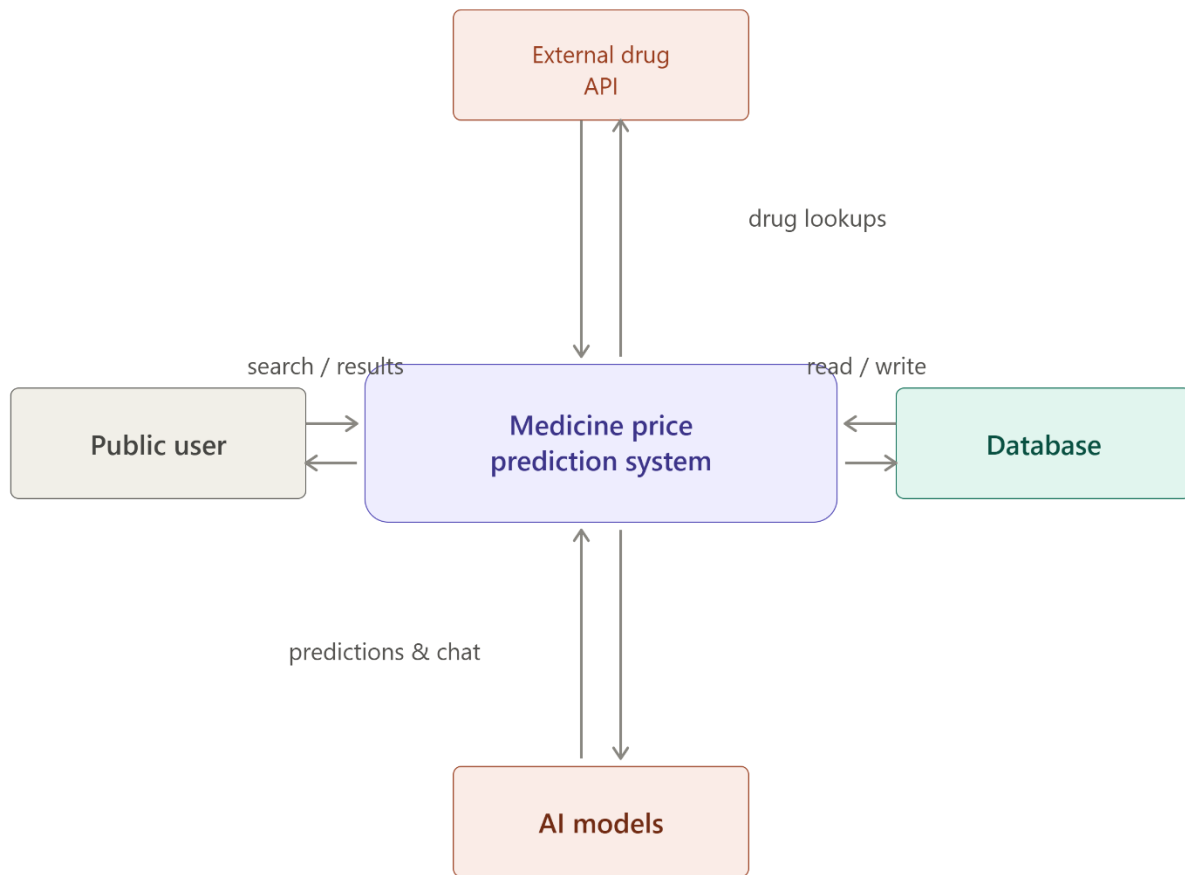
This design offers several advantages:

- Preserves the complete price history for every medicine.
- Enables accurate visualization of price trends over time.
- Provides chronological data required for AI-based forecasting models.
- Supports efficient retrieval of historical records through indexed timestamp fields.
- Allows new price records to be appended without modifying previous data, ensuring data integrity and maintaining an immutable historical dataset.

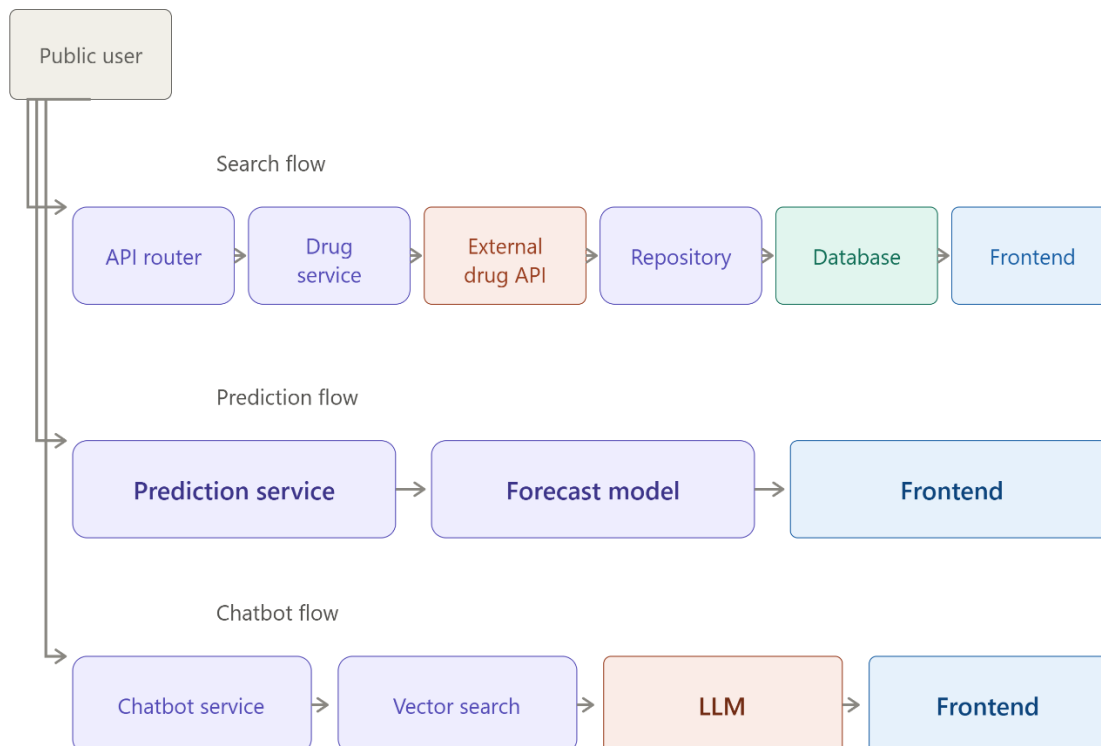
Separating historical prices into a dedicated table makes the database well suited for time-series analytics and future machine learning or AI-based prediction models while maintaining high query performance and scalability.

4.8 Data Flow & System Behavior

DFD Level 0

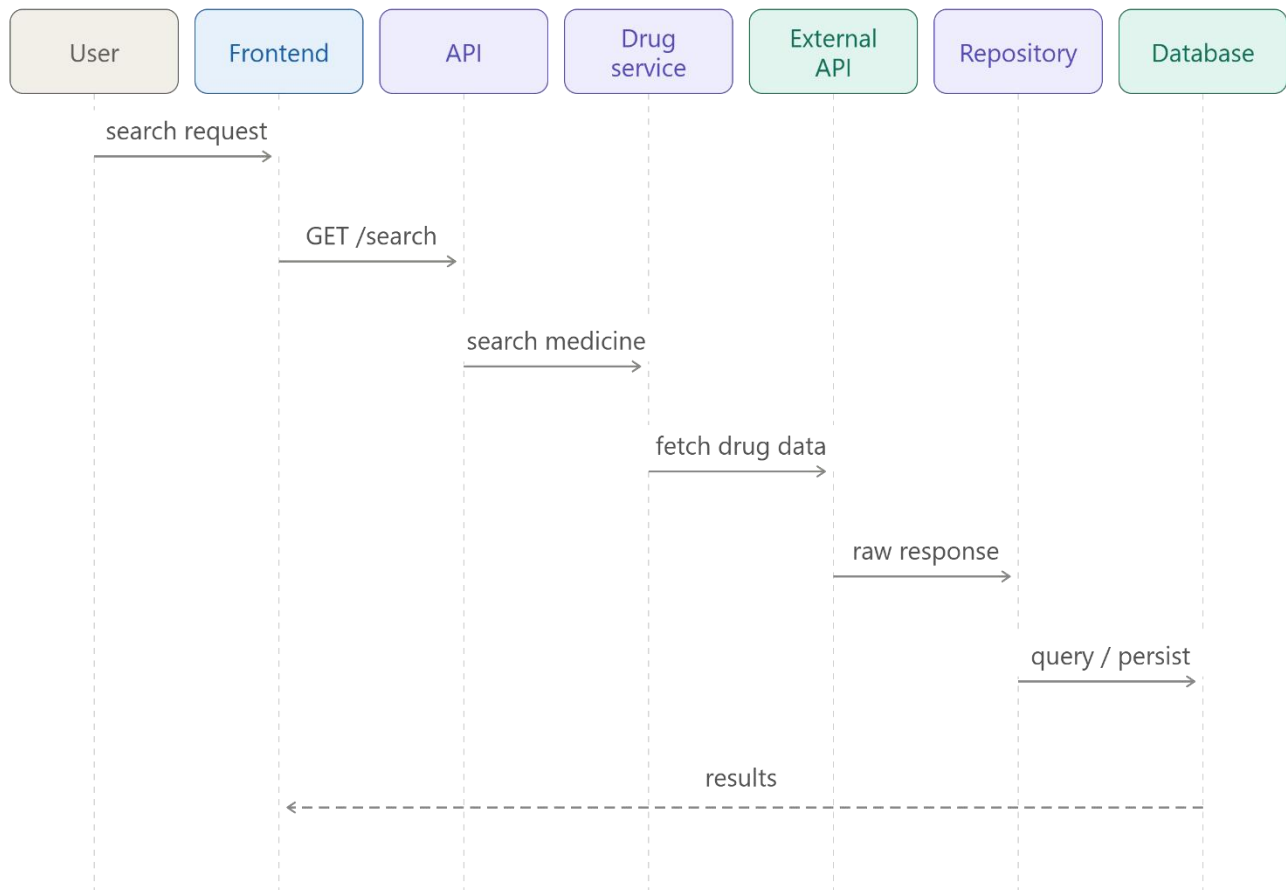


DFD Level 1

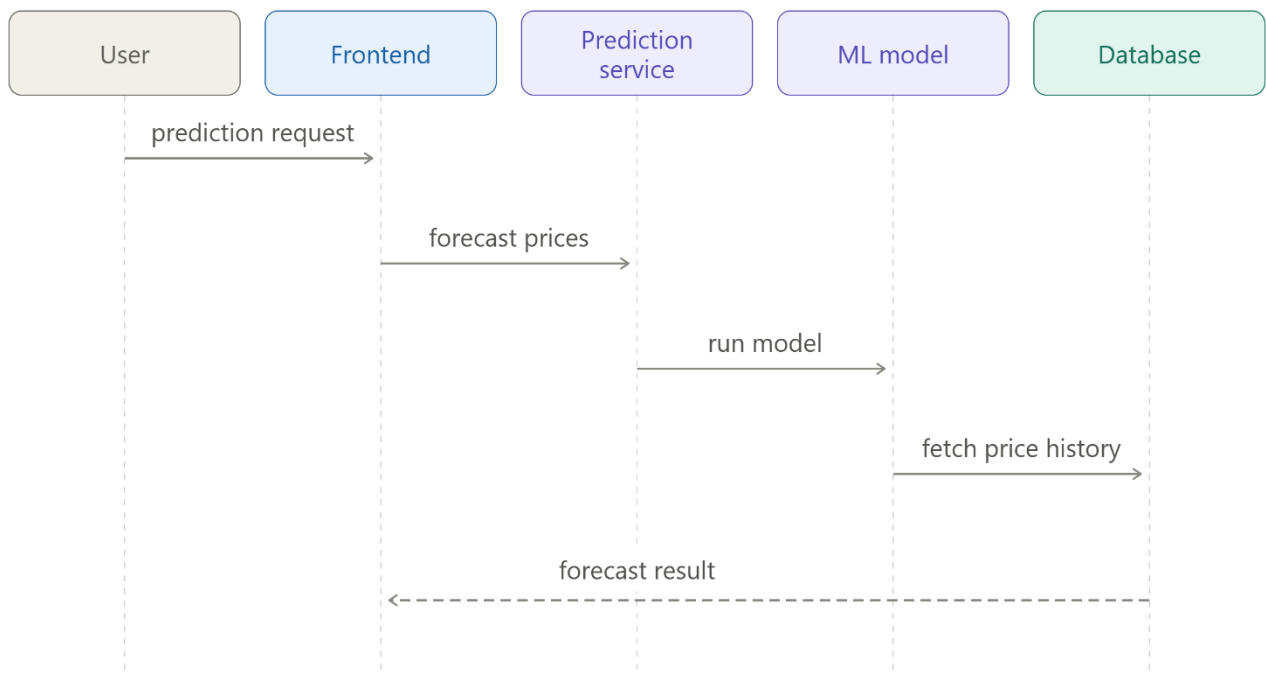


4.9 Sequence Diagram

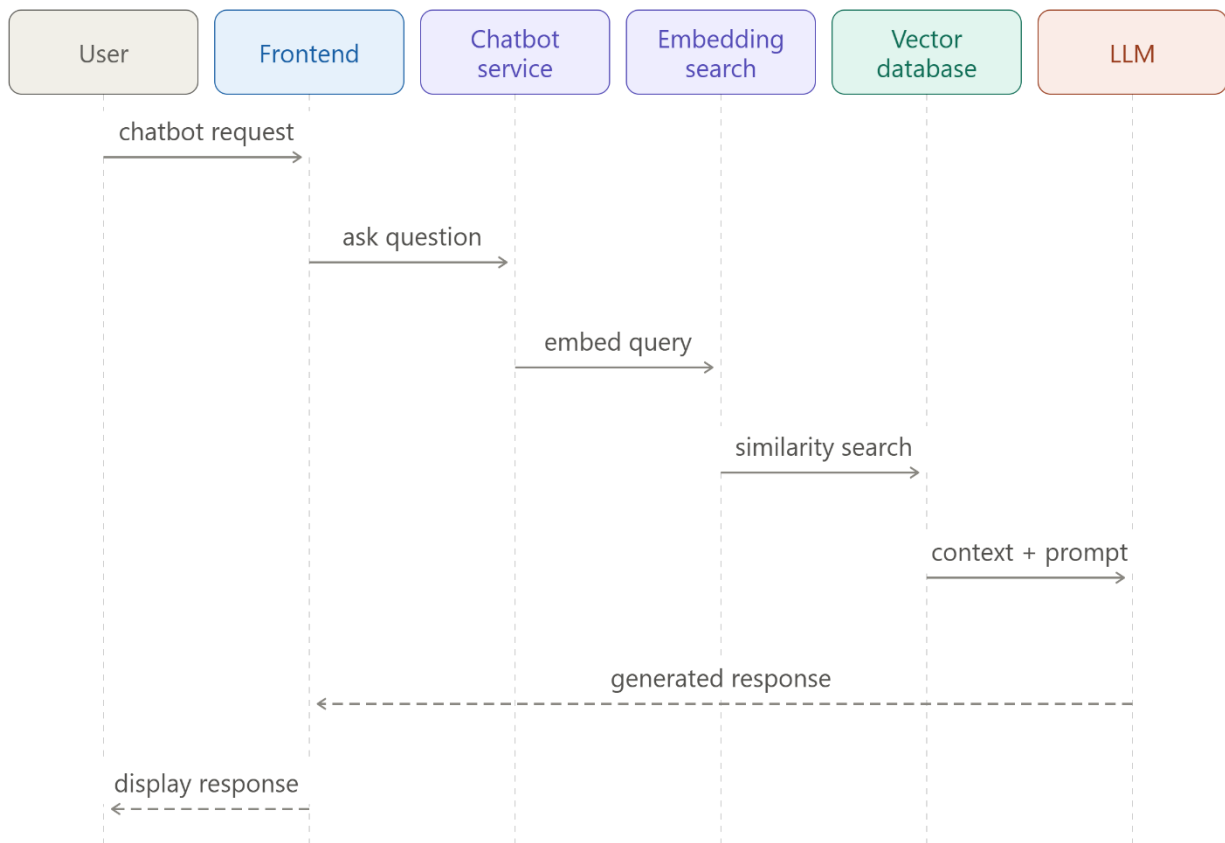
Show the interaction for a medicine search:



Prediction:

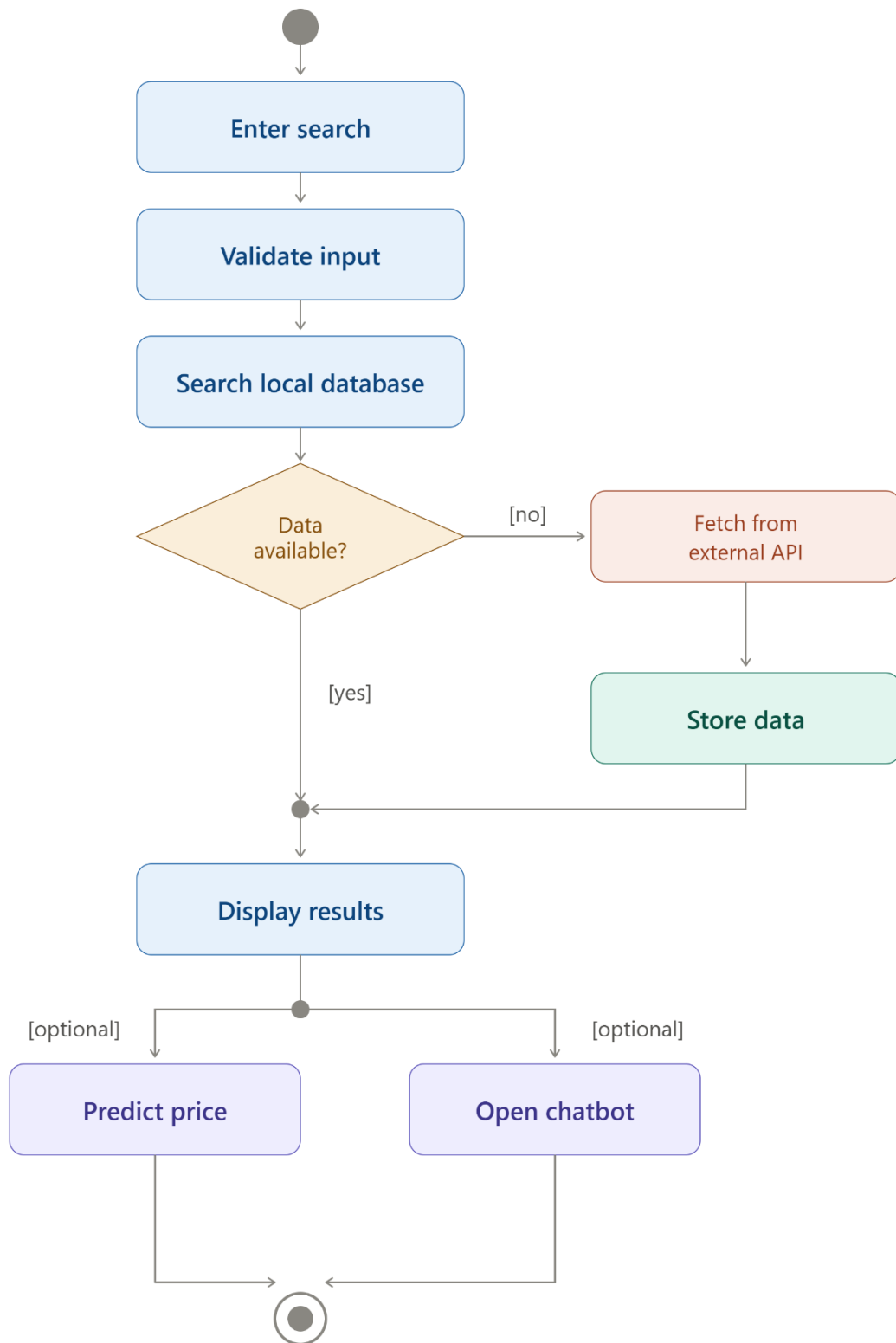


Chatbot:



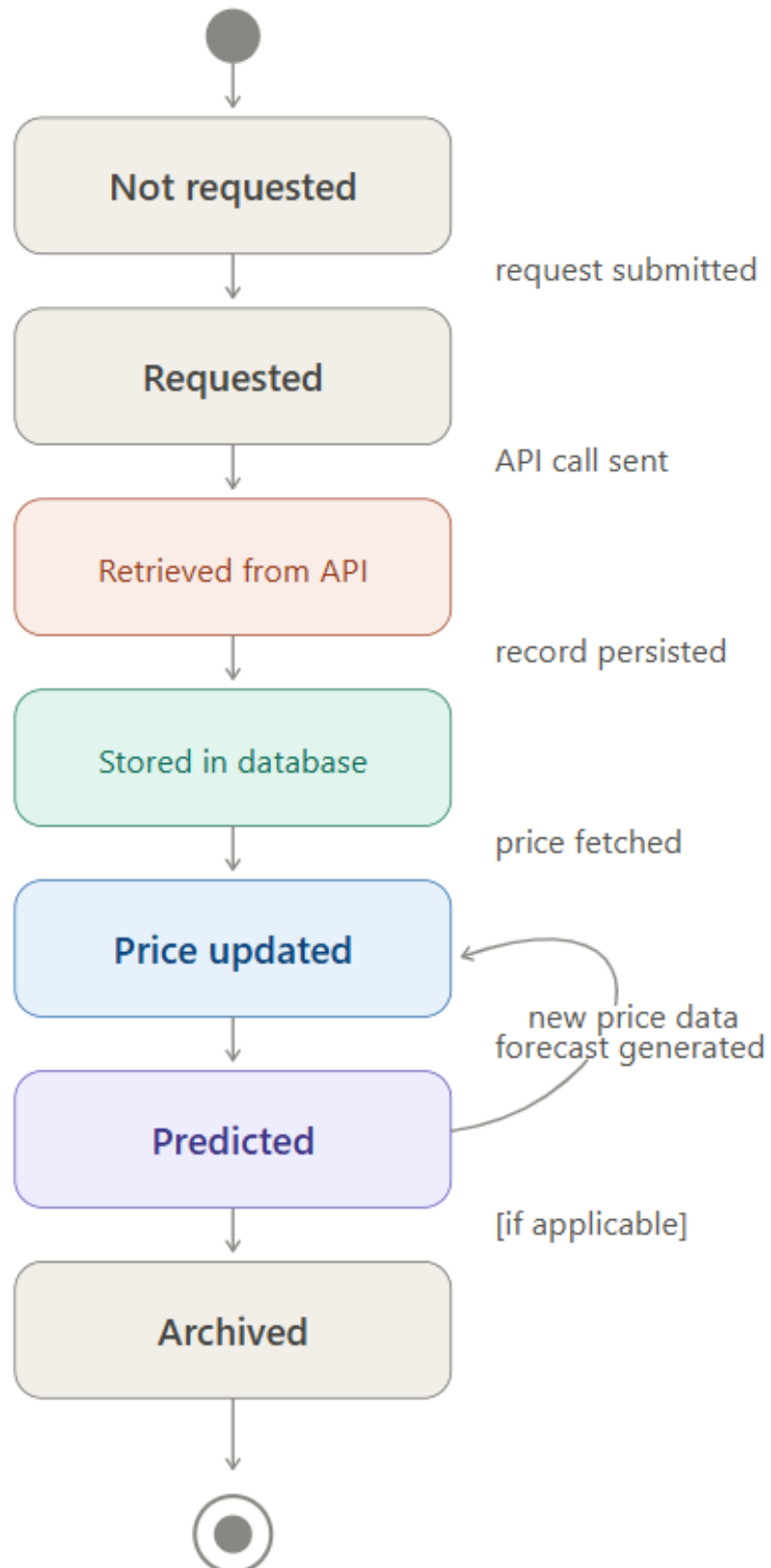
4.10 Activity Diagram

Illustrates the workflow:

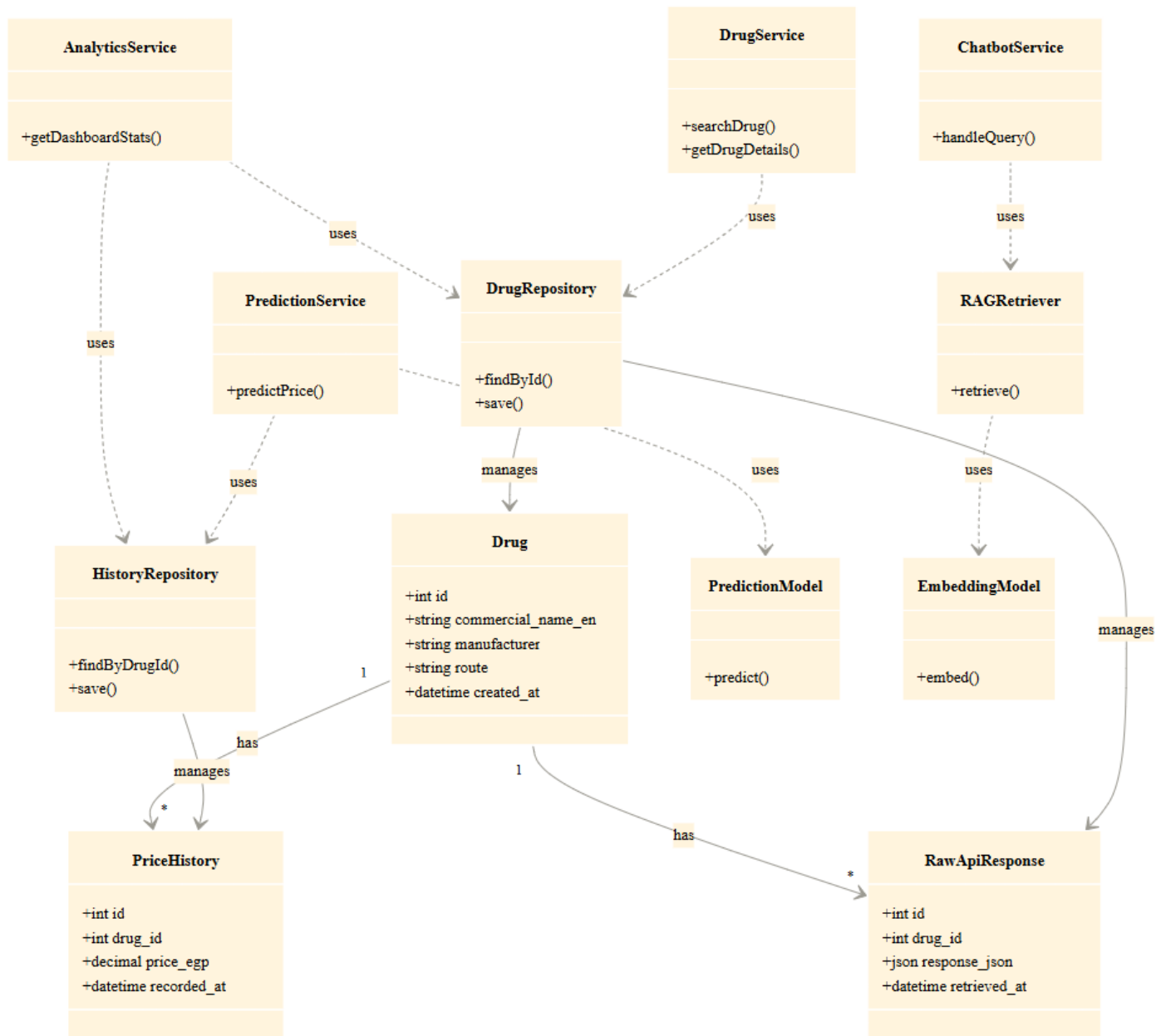


4.11 State Diagram

Model the lifecycle of a medicine record:



4.12 Class Diagram



4.13 UI/UX Design & Prototyping

Medicine Predictor

DashboardAlternativesAlertsAnalytics

Medicine Dashboard

Search, filter and monitor medicines available in the Egyptian database.

Medicines
10

Average Price
34.90 EGP

Manufacturers
6

Expensive Medicines
0

Commercial Name	Scientific Name	Manufacturer	Drug Class	Route	Price
1 2 3 (ONE TWO THREE) 20 F.C.TABS.	CHLORPHENIRAMINE+PARACETAMOL(ACETAMINOPHEN)+PSEUDOEPHEDRINE	HIKMA PHARMA	COLD PRODUCTS	ORAL.SOLID	10 EGP
1 2 3 (ONE TWO THREE) EXTRA 20 F.C.TABS.	CHLORPHENIRAMINE+PARACETAMOL(ACETAMINOPHEN)+PSEUDOEPHEDRINE	HIKMA PHARMA	COLD PRODUCTS	ORAL.SOLID	64 EGP
1 2 3 (ONE TWO THREE) SUSP. 120 ML	CHLORPHENIRAMINE+PARACETAMOL(ACETAMINOPHEN)+PSEUDOEPHEDRINE	HIKMA PHARMA	COLD PRODUCTS	ORAL.LIQUID	7 EGP

Alternative Medicines

Search medicines and explore alternative drugs available in the database.

Filters

Available Medicines50

1 2 3 (ONE TWO THREE) 20 F.C.TABS.

1 2 3

CHLORPHENIRAMINE+PARACETAMOL(ACETAMINOPHEN)+PSEUDOEPHEDRINE

HIKMA PHARMA

10.00 EGP

View Details

1 2 3 (ONE TWO THREE) EXTRA 20 F.C.TABS.

1 2 3 [كسترا]

CHLORPHENIRAMINE+PARACETAMOL(ACETAMINOPHEN)+PSEUDOEPHEDRINE

HIKMA PHARMA

64.00 EGP

View Details

1 2 3 (ONE TWO THREE) SUSP. 120 ML

1 2 3

CHLORPHENIRAMINE+PARACETAMOL(ACETAMINOPHEN)+PSEUDOEPHEDRINE

HIKMA PHARMA

7.00 EGP

View Details

ADVIPRO COUGH & COLD SYRUP 120ML

ALLERGYL 2% TOPICAL CREAM 15 GM (N/A)

ALLERGYL 2MG/5ML SYRUP 120ML

Alerts

Important notifications about medicine prices and predictions.



Price Increase Expected

Today

Panadol is predicted to increase by 15% next month.



High Price Alert

Yesterday

Augmentin exceeded its average market price.



New Prediction Available

2 days ago

Predictions have been updated successfully.

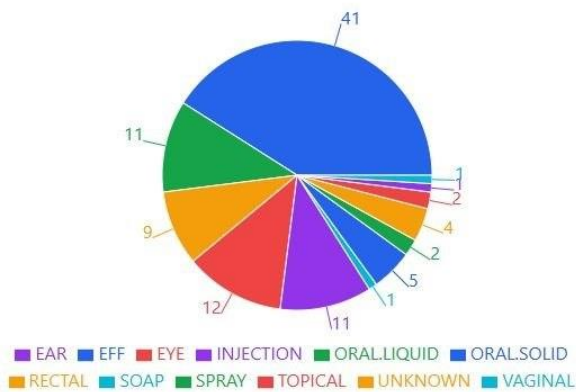


Alternative Found

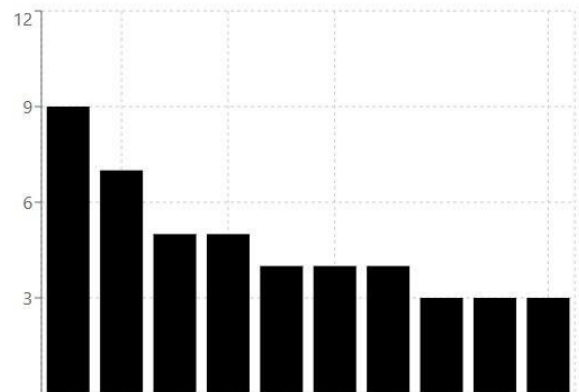
3 days ago

A cheaper alternative was found for Voltaren.

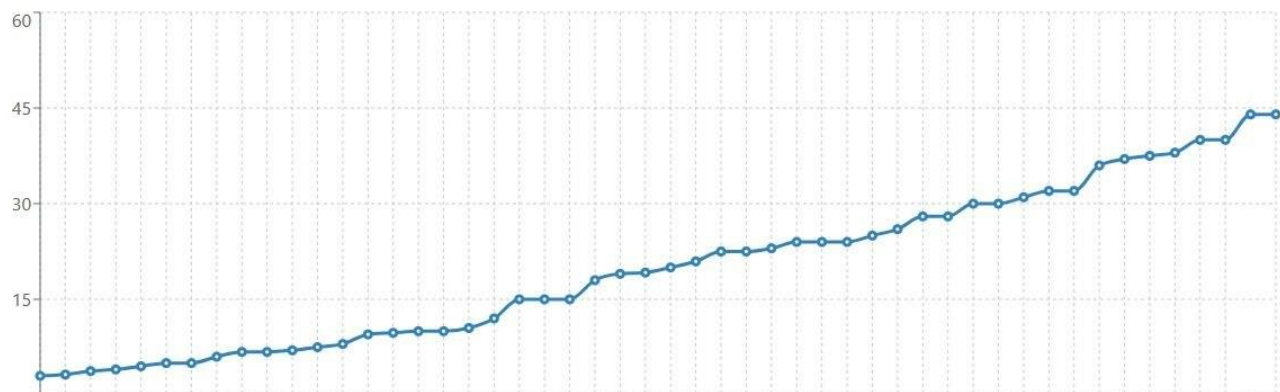
Drug Routes



Top Drug Classes



Price Distribution

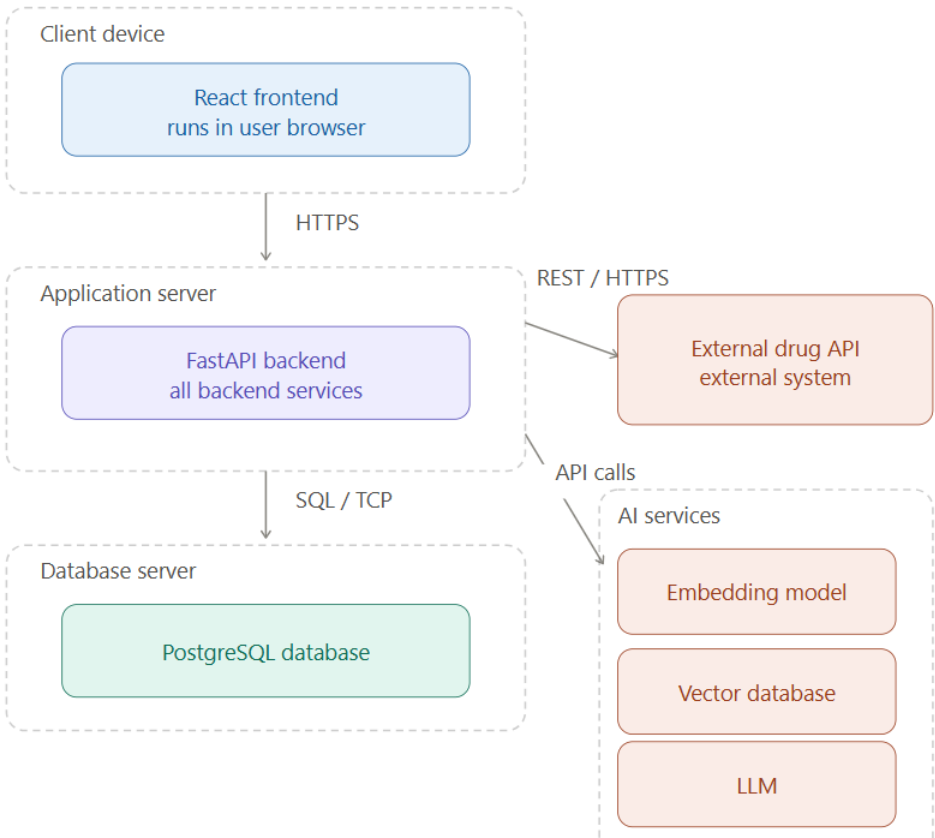


4.14 System Deployment & Integration

Technology Stack

Layer	Technology
Frontend	React, Vite
Backend	FastAPI (Python)
Database	PostgreSQL
ORM	SQLAlchemy
ML Libraries	Pandas, Prophet (price forecasting), RAG pipeline
AI Components	Embedding Model, Vector Database, LLM
HTTP Client	HTTPX
Charts	Recharts
State Management	Redux Toolkit
Styling	Tailwind CSS
API Documentation	Swagger / OpenAPI

Deployment Diagram



Component Diagram

